

CICADA 2067

UNIFIED DATABASE SCHEMA & ARCHITECTURE SPECIFICATION

This technical specification outlines the unified database schema blueprint for the **Cicada 2067** platform. This model merges previous high-level architectures with strict decoupled progress-tracking layers. Specifically, it implements an explicit interval-based duration model to trace participant progression using discrete opening and completion timestamps.

Architectural Core Change: The timeline framework has shifted from a simplified differential timestamp calculation to a strict, high-fidelity `opened_at` and `completed_at` state validation system. This structural change guarantees audit-ready timeline tracking, precise rate-limiting, and clean handling of global event pauses.

1. Relational Database Schema (PostgreSQL DDL)

Global Custom Types

```
CREATE TYPE user_role AS ENUM ('participant', 'admin');
```

1. Users Table

Serves as the pre-authorized whitelist and user profile workspace. A PostgreSQL trigger tied to Supabase Auth verifies registering emails against this table before validating user account deployment.

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  display_name VARCHAR(255),  
  role user_role DEFAULT 'participant' NOT NULL,  
  team_id UUID REFERENCES teams(id) ON DELETE SET NULL,  
  joined_team_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

2. Teams Table

Created exclusively by a designated Team Leader. Generates a unique secure invitation code for onboarding up to a maximum of 5 members per team.

```
CREATE TABLE teams (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL UNIQUE,  
  leader_id UUID REFERENCES users(id) ON DELETE SET NULL,  
  invite_code VARCHAR(100) UNIQUE NOT NULL,  
  is_disqualified BOOLEAN DEFAULT FALSE NOT NULL,
```

```

    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

3. Challenges Table

Unified data entity containing static puzzle parameters, storylines, asset path maps, and cryptographic answer verifications.

```

CREATE TABLE challenges (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    challenge_name VARCHAR(255) NOT NULL,
    story_context TEXT NOT NULL,
    embedded_assets JSONB DEFAULT '[]'::jsonb, -- Structured map array of asset file paths
    answer_hash VARCHAR(255) NOT NULL,         -- Bcrypt/Argon2 cryptographic hash
    verification
    unlock_sequence INT NOT NULL UNIQUE,       -- Strict sequential index mapping challenge
    order
    unlocks_story_fragment TEXT,               -- Textual narrative lore unlocked upon
    completion
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

4. Team Progress Table UPDATED

The decoupled operational hub optimized for real-time leaderboard aggregation and live progress diagnostics.

```

CREATE TABLE team_progress (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE CASCADE UNIQUE,
    current_challenge_id UUID REFERENCES challenges(id) ON DELETE SET NULL,
    challenges_completed INT DEFAULT 0 NOT NULL,
    total_time_taken INT DEFAULT 0 NOT NULL,    -- Accumulated competition duration
    in seconds
    current_challenge_attempts INT DEFAULT 0 NOT NULL, -- Counter utilized for brute-force
    rate-limiting
    -- High-Precision Timing Anchors
    opened_at TIMESTAMPTZ DEFAULT NOW(),       -- Timestamp marking access provision
    to the puzzle
    completed_at TIMESTAMPTZ,                  -- Timestamp marking the successful
    submission log
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

5. Unlocked Story Fragments (Bridge Entity)

Links achievements directly to accessible lore metadata, ensuring persistent history log retrieval for participant client dashboards.

```
CREATE TABLE unlocked_story_fragments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
  challenge_id UUID REFERENCES challenges(id) ON DELETE CASCADE,
  unlocked_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(team_id, challenge_id)
);
```

6. Submission Logs

Immutable ledger records capturing all interactions for security telemetry, cheating mitigation, and administrative verification loops.

```
CREATE TABLE submission_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  challenge_id UUID REFERENCES challenges(id) ON DELETE SET NULL,
  submitted_answer TEXT NOT NULL,
  is_correct BOOLEAN DEFAULT FALSE NOT NULL,
  submitted_at TIMESTAMPTZ DEFAULT NOW()
);
```

7. System Config Table

Global runtime key-value maps isolated for Administrative controls (e.g., executing global platform lockdowns or pausing active timers).

```
CREATE TABLE system_config (
  key VARCHAR(255) PRIMARY KEY,
  value JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

2. Precision Duration & Timeline Tracking Model

The time calculation logic leverages isolated state updates on the `team_progress` framework. By utilizing strict `opened_at` and `completed_at` anchors, the system maintains chronological accuracy across edge network conditions.

A. Mathematical Core Formula

When a participant submits an answer verified against the cryptographic `answer_hash`, the specific challenge solve duration (Δt) is computed immediately as follows:

$$\Delta t = \text{completed_at} - \text{opened_at}$$

The global duration ledger index is then updated in real time by appending the newly calculated delta value to the running aggregate:

$$total_time_taken_{new} = total_time_taken_{old} + \Delta t$$

B. Lifecycle Operations

Lifecycle Phase	Database Execution Logic	State Mutated
Challenge Opening	Executed when a team accesses a puzzle block. Writes the exact server epoch into the start parameter.	<code>opened_at = NOW()</code> <code>completed_at = NULL</code>
Verification & Log	Fires upon validated answer verification. Anchors the termination matrix timestamp.	<code>completed_at = NOW()</code>
Delta Aggregation	Calculates interval difference in seconds and increments progress records. Clears attempts.	<code>total_time_taken += Δt</code> <code>challenges_completed += 1</code> <code>current_challenge_attempts = 0</code>

3. Security & Isolation Architecture

Security perimeters isolate standard team environments from administrative control surfaces. This is governed directly through Supabase metadata filtering and PostgreSQL Row Level Security (RLS) layers.

- **Participant Isolation** (`role = 'participant'`): Users are restricted to horizontal read permissions. They can only query rows from `users` matching their own `team_id`. Under `challenges`, they can strictly view the single active record where `challenges.id = team_progress.current_challenge_id`. Crucially, the sensitive `answer_hash` field is isolated on the database level and never exposed to client queries; comparison takes place entirely within an authenticated RPC/Edge execution layer.
- **Administrative Operations** (`role = 'admin'`): Systems administrators completely bypass RLS parameters. They maintain comprehensive read, write, update, and drop permissions to support manual workflow changes, track live threat intelligence, and configure the global state ledger.